System overview

The client's Material Management SOP — 22 steps from a supplier's vehicle at the gate to finished goods leaving the gate — turned from a browser-only demo into a secure, multi-user web application with a real database. This document explains what was built, why, and how the logic works.

	Where it started	Where it is now
Data	Kept in the browser's memory — gone on refresh, one copy per tab	MySQL 8, relational; shared live by every department
Logins	Passwords inside the web page	Server-side accounts, hashed passwords, secure sessions, lockout
Rules	Checked only on screen	Enforced on the server and, for passed stages, by the database itself
Many users	Not possible	Parallel work; conflicts detected; no double stock; gap-free numbers
Traceability	In the browser only	Every document, every attempt and every login in an append-only audit log

At a glance

Built	Count / detail
Register tables (relational)	18 business tables + 9 master tables + 7 system tables, 1 view
Database migrations	6 — each applied once, in order, recorded in schema_migrations
Stage-lock triggers	40 — passed stages cannot be altered, registers cannot be deleted
Actions (each permission-checked, validated, locked, audited)	19 register actions + customer edit + protected demo reset
Automated checks	104 across 5 suites — security, parallel use, data integrity, reset protection, concurrency

1. Architecture

One server hosts both the screens and the API on the same domain. That keeps the login cookie strict and removes cross-site configuration. Every request passes the same chain of checks; the business rules are written once and shared by the server (which enforces them) and the browser (which only displays figures).

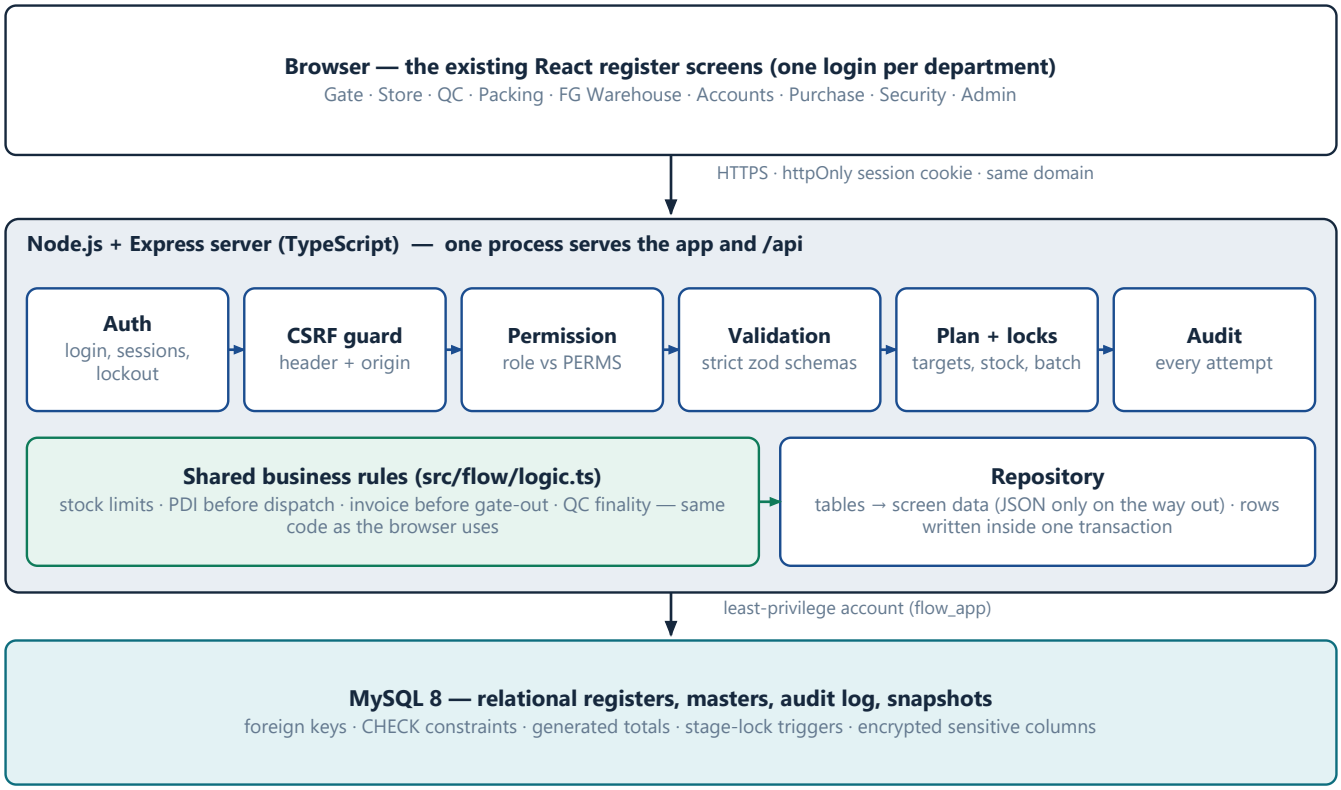


Figure 1 — the parts of the system and what each one is responsible for

Why this shape

- **The screens stayed as they were** — users already know them and they mirror the client's paper forms. Only the data layer underneath changed.
- **One action endpoint** (`POST /api/actions/<name>`) instead of dozens of URLs: every change goes through the same permission → validation → lock → transaction → audit path, so no URL can forget a check.
- **Shared rules** (`logic.ts`): the rules are not rewritten for the server — the same functions run there, inside a transaction, and the server decides.
- **TypeScript** on both sides, compiled to plain JavaScript for production — a renamed field breaks the build instead of the factory floor.

2. The material flow — who does what

Each box is a register action; the label above it is the login that performs it. The arrows follow the material. Every step is a document with its own number, and every step checks the one before it.

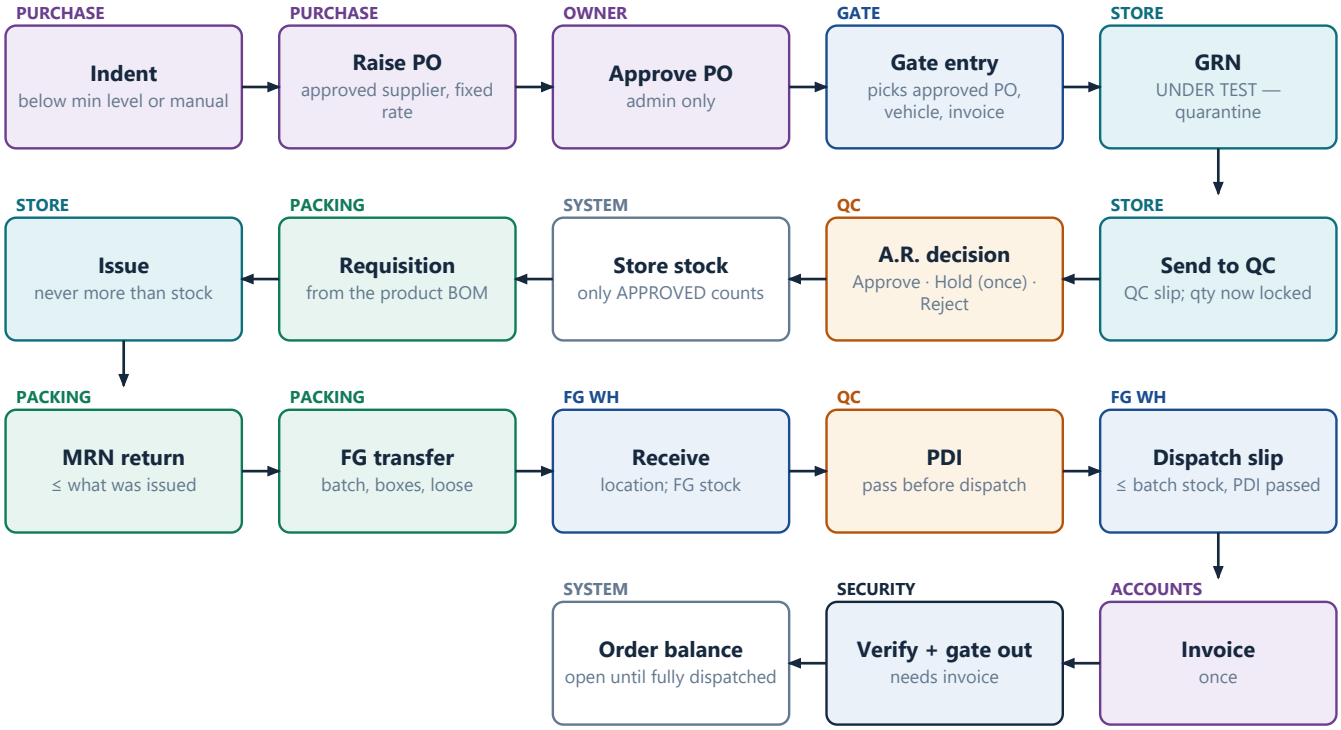


Figure 2 — the 22-step SOP as registers and logins (read left-to-right, then right-to-left)

Department login	Can do	Can view (read-only)
Gate	Gate entry (picks approved PO)	—
Store	GRN, quantity correction until sent to QC, send to QC, barcode, issue, MRN	—
QC	A.R. decision (approve / hold once / reject), PDI	GRN
Packing	Requisition, FG transfer, MRN	QC
FG Warehouse	Receive transfers, customer orders, dispatch slips	—
Accounts	Invoice on dispatch, customer orders, edit customer master	—
Purchase	Indents, purchase orders	PM stock
Security	Verify and gate out	Gate register
Admin (Owner)	Everything above + approve POs; demo reset only where switched on	—

Every login lands on its own pending work (e.g. QC: materials waiting for a decision; Security: vehicles awaiting gate-out). Each role receives only the registers its screens need; prices and customer details are blanked for roles that have no business seeing them.

3. What happens when someone clicks Save

Every change — a gate entry, a QC decision, an issue, a dispatch — goes through these steps on the server. If any check fails, nothing is saved and the user sees the reason (red boxes).

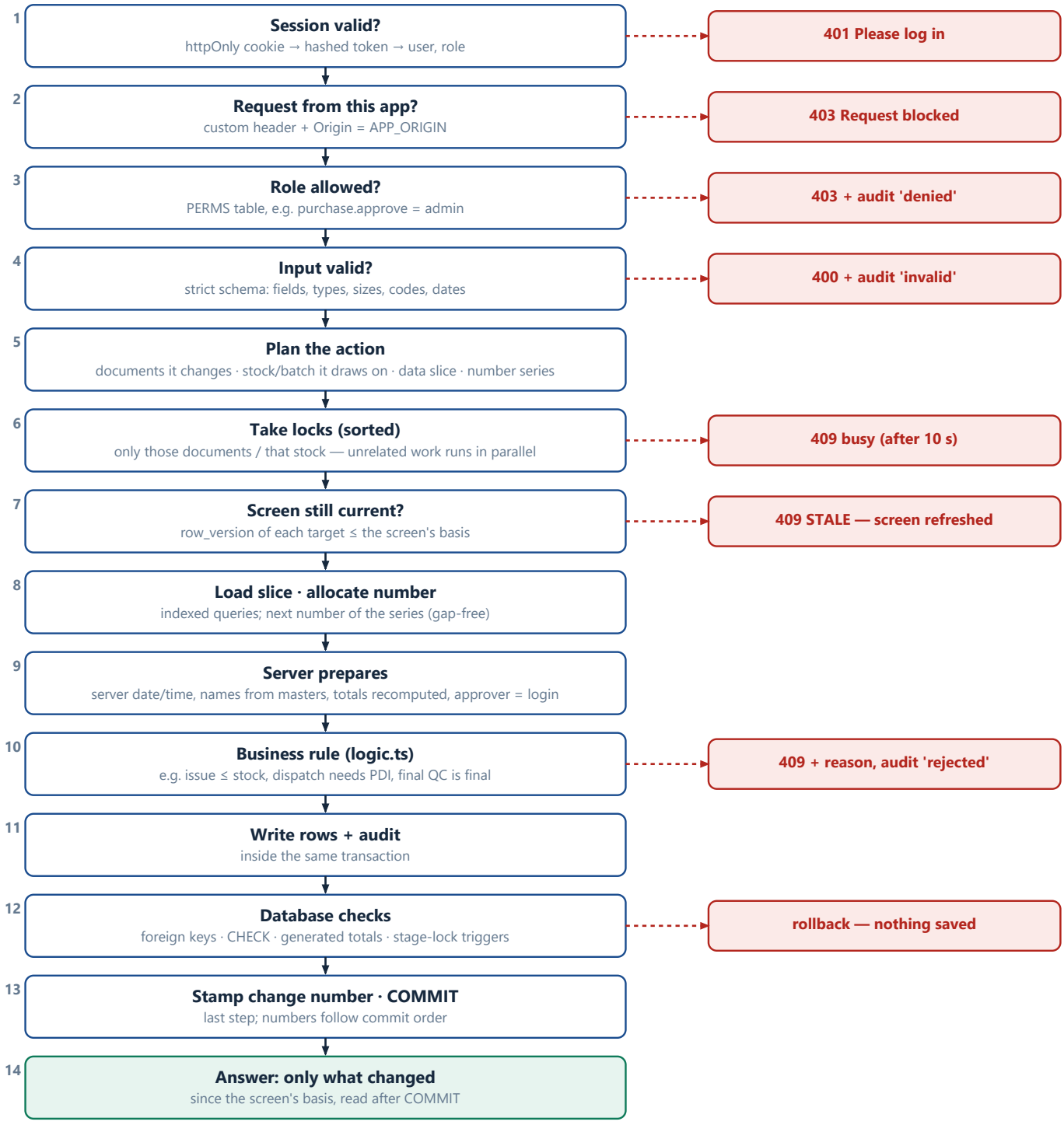


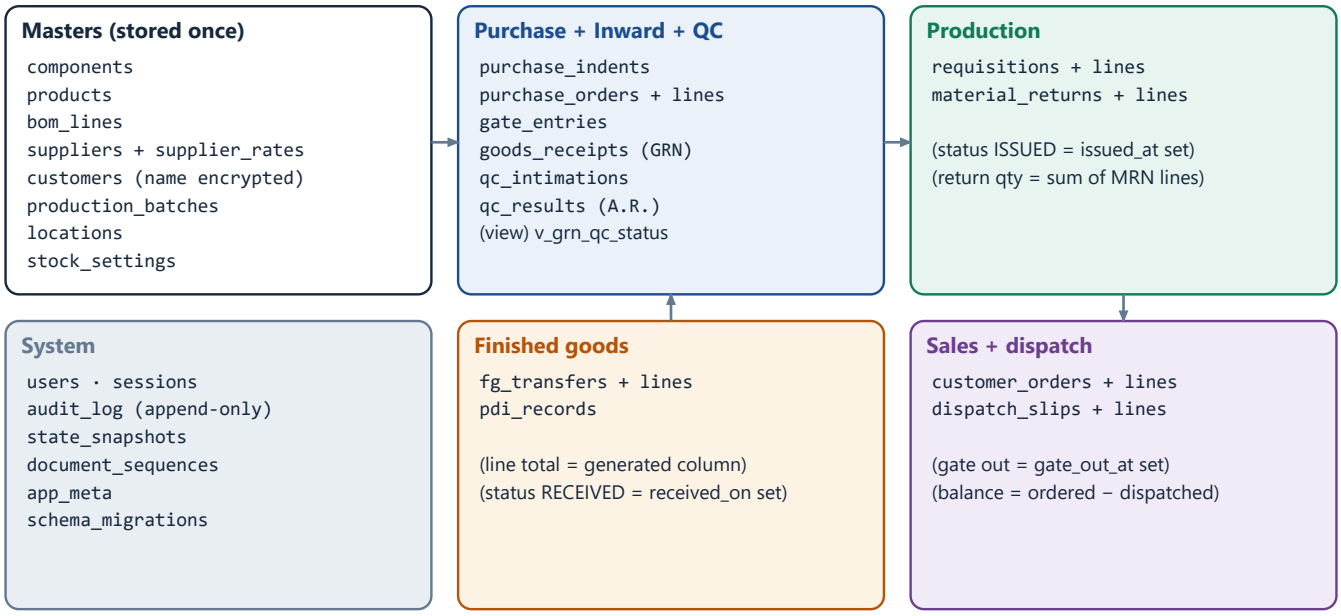
Figure 3 — the life of one action (steps 6–13 run inside one database transaction)

4. Security — layer by layer

Concern	What protects it
Passwords	scrypt hashes with a per-user salt; policy ≥ 10 characters and 3 character classes; temporary passwords must be changed at first login; never logged
Sessions	256-bit random token in an httpOnly, SameSite=Strict, Secure cookie — scripts on the page cannot read it; only its SHA-256 is stored; 2 h idle / 12 h maximum; new token on every login; password change signs out other devices
Password guessing	5 wrong passwords lock the login for 15 minutes (atomic counter); 20 failed logins per network per 15 minutes; unknown and real users get the same answer in the same time
Forged requests	strict cookie + a required custom header + Origin must equal the app's address; cross-site access (CORS) is never enabled
Who may do what	every action checked on the server against the permission table; hidden buttons are only cosmetic
Who may see what	each role gets only its registers; PO rates and supplier prices only for Purchase/Admin; customer names, addresses and invoice numbers only for FG WH, Accounts, Security, Admin; product recipes only for roles that raise requisitions — nothing is compiled into the public web page
Bad input	strict schemas: unknown fields, wrong types, oversized values, unknown codes and bad dates are refused
Tampering with facts	the server sets dates/times, names, totals and the approver; the browser cannot back-date or inflate
Encryption at rest	AES-256-GCM: customer name/address/PO, invoice numbers, PO and supplier rates, vehicle and LR numbers, audit details, snapshots
Database account	table-level privileges only; no DELETE on registers on the live system; masters read-only; audit log insert-only
Destructive reset	only where switched on, admin only, admin's password + typed phrase, rate-limited, automatic encrypted snapshot first
Audit	every login, logout, password change and action — allowed, refused, denied or invalid — with user, role, IP and time
HTTP	strict Content-Security-Policy, HSTS, no framing, no-sniff; API answers never cached; errors never reveal internals

5. Database — each fact stored once

Every register is its own table with typed columns; document lines are child tables. Documents point to masters by key, so renaming a product, component, supplier or customer changes one row and every document shows the new name at once. Nothing derived is stored.



Every document refers to masters by key (code / id) — names live only in the master tables.

Statuses, line totals, stock and balances are computed, never stored twice.

Figure 4 — table groups and how they refer to each other

Shown on screen	Where it actually lives
Supplier, invoice and PO on a GRN	the GRN's gate entry
Component name on any document	components (by code)
Quantity on a QC slip	the GRN it was raised for
QC status of a GRN	its latest A.R. result (database view)
Returned quantity on a requisition	the sum of its MRN lines
Customer name and address on orders/dispatches	customers (encrypted), found by a keyed hash
Line totals (boxes × per box + loose)	computed by MySQL itself (generated column)
Store stock, FG stock, PO pending, order balance	computed from the documents — never a second ledger

Conversion of the existing data was verified before the old storage was dropped: document counts, store stock, FG stock, PO and order balances had to match exactly, or nothing was kept.

6. Passed stages are locked

Once a document passes a stage, the facts of that stage cannot be altered — enforced twice: by the business rules (with a clear message) and by MySQL triggers (so not even a bug, a direct SQL edit or a compromised app account can alter them).

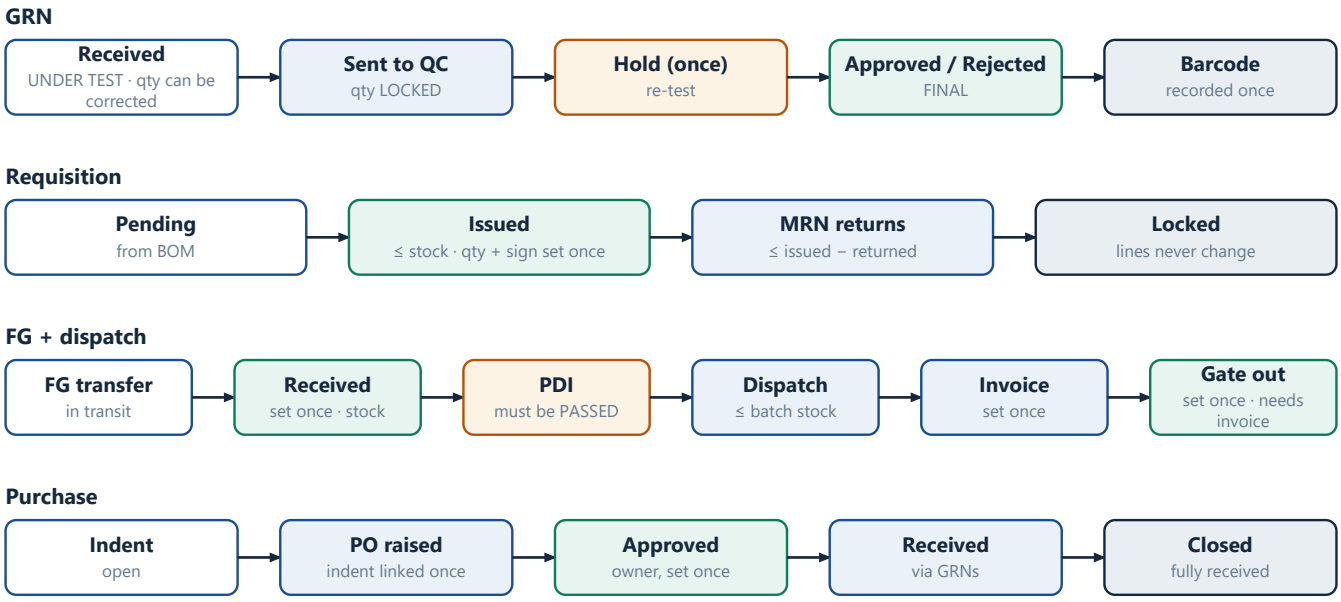


Figure 5 — document lifecycles; every arrow is one-way

Rule	Applies to
Written once, never edited	gate entries, QC slips, QC results, MRNs, PDI, customer orders, every document line, PO lines
Filled in once, then frozen	PO approval, indent → PO link, issued quantities and store sign, FG receipt, invoice, gate-out
Locked from a stage on	GRN quantity once sent to QC
Never deleted	register entries (only the protected demo reset, where switched on)
Append-only for every account	audit log and safety snapshots — even the database administrator

Corrections after a stage need a cancelling / reversing document with a reason — planned, not yet built.

7. Many people at the same time

Locks only where needed. An action locks just the documents it changes, the stock of the components it issues, or the FG batch it ships or inspects. Admin approving a PO and QC approving a GRN never wait for each other; two issues of the same component take turns, so stock can never be overdrawn. Locks are always taken in the same order, so two actions can never wait on each other forever.

Stale screens are refused. Every document carries the change number of its last change; every action carries the change number its screen was built from. If a document it touches moved on since, the action is refused and the screen refreshes:

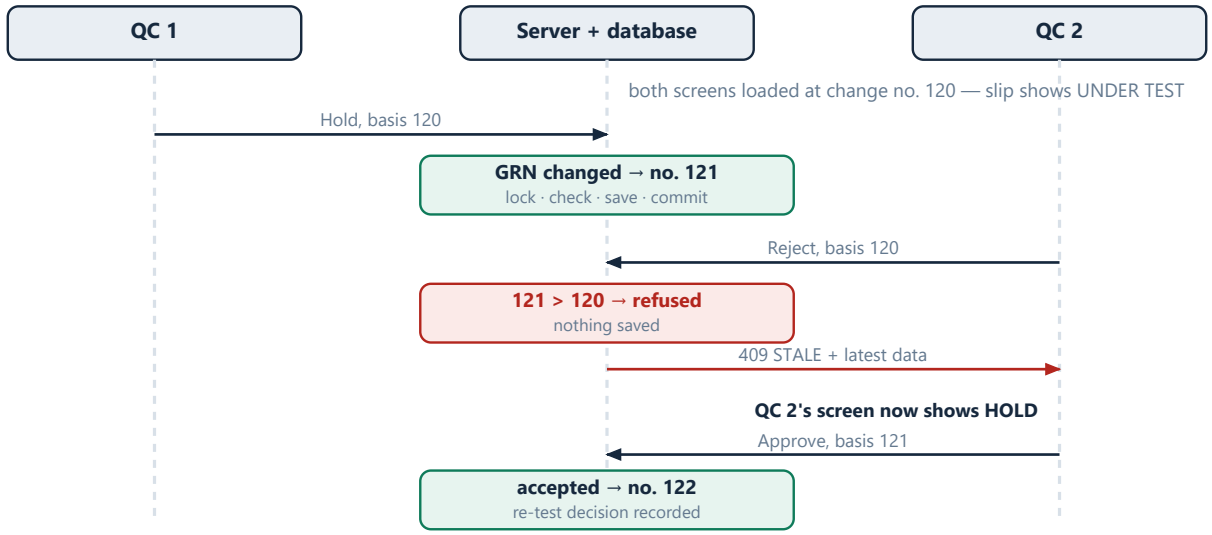


Figure 6 — two QC screens act on the same slip at the same moment

Numbers never collide or skip. Each document series (GE, GR, QCI, …, PO) has its own sequence row, locked only while one document of that series is created; a failed action gives its number back.

Screens stay in sync cheaply:

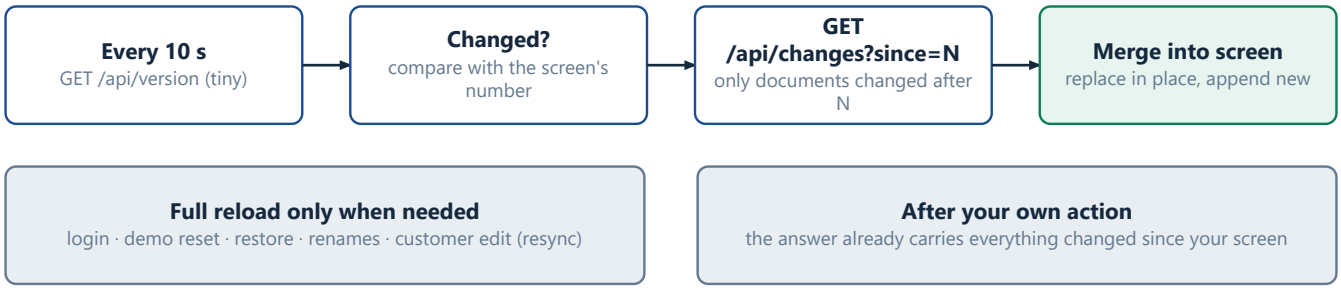


Figure 7 — how each screen keeps up with the other departments

8. How it is verified

Five automated suites run against a real server and database. All pass: **104 checks**.

Suite	Checks	What it proves
security	31	login, lockout, forged requests, role limits on actions and on data, server-set dates/totals, over-issue and over-dispatch refused, gate-out needs invoice, logout
parallel	15	GRN quantity locked after Send to QC; stale QC decision refused and recovered; hold only once; final QC decision final; barcode once; only changed documents sent; 12 simultaneous saves with gap-free numbers; a busy document does not block another
integrity	36	product / supplier / customer renames consistent everywhere; customer edit guarded; MySQL rejects broken references, impossible values and every stage-lock violation; even the database admin cannot edit the audit log; failed transactions leave nothing behind
reset	11	no one-click wipe; wrong phrase or password refused; non-admin refused; snapshot taken
concurrency	11	parallel GRNs unique; two parallel issues cannot overdraw; lockout; password change signs out other sessions; forged token refused

9. Going live

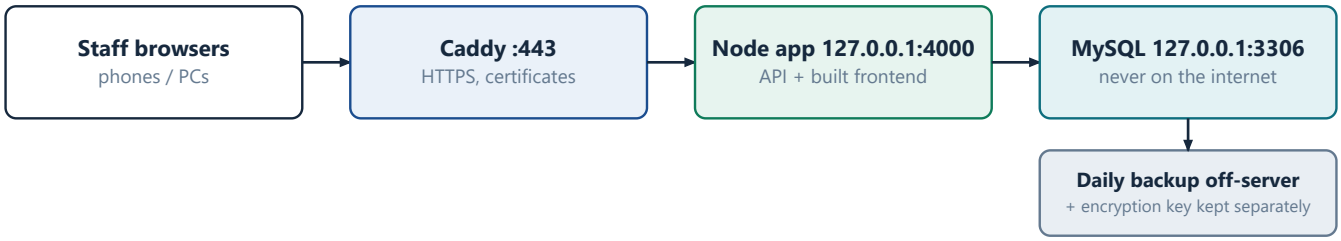


Figure 8 — production layout on one small server in India

Step	Command / action
Configure	npm run env:init -- --production → set APP_ORIGIN (https://...) and database admin in .env
Database	npm run db:setup → tables, triggers, restricted app account, logins with one-time passwords
Build + run	npm run build · npm start behind Caddy (HTTPS); keep ALLOW_DEMO_RESET=false
Harden	remove the DB admin from .env, chmod 600 .env, only ports 80/443 open
Backups	daily mysqldump off the server; keep DATA_ENCRYPTION_KEY separately; test one restore

10. What was built, in order — and why

#	Built	Why
1	Express backend, MySQL, logins, permissions, validation, audit, encryption	Make the demo a real multi-user system that is secure from the start
2	Protected reset: password + phrase + snapshot	A one-click wipe of every department's data was too dangerous
3	Relational schema; names in masters only; JSON only at the API	Renames must stay consistent; no copy of a fact may disagree with another
4	Customer master with safe editing	Name/address corrections apply everywhere; no lost updates
5	Stale-screen guard, per-document locks, number sequences, change-only sync	Departments act at the same time without waiting or overwriting each other
6	Stage locks in rules and database triggers	A passed stage must never be altered — whoever tries

Next steps

- **Fit into the client's Google Forms / Sheets flow**: map each form to its register, import masters and the go-live opening position, pilot Gate + Store, then switch departments over one by one and close each form.
- **Order reservation**: show available stock at order entry and reserve on save (back-order when short) — awaiting the client's policy.
- **Cancel / reverse documents** with a reason — the correct way to correct a passed stage.
- **Lot-level traceability**: allocate each issue to specific GRN lots, so a defective supplier lot leads to the exact FG batches.
- **Gate completeness**: inward vehicle time-out; outward vehicle, driver and transporter on dispatch.
- **Purchase fixes**: count only QC-approved quantity against a PO; flag over-delivery; capture supplier lot, MFG and expiry at Send to QC.
- **Growth**: server-side stock figures and page-by-page history for many years of data; printable forms.